Программная инженерия. Лекция 4
Construction: CI/CD и тестирование

Михаил Пожидаев

19 февраля 2026 г.

Цель и задачи

Движемся к бета-версии

Главная цель — начальная работоспособность, что подразумевает создание продукта, готового к бета-тестированию. Для этого нужно:

- расширить определение вариантов использования и классов до модели реализации;
- выполнить реализацию;
- организовать обслуживание процесса сборки с применением практик CI/CD;
- организовать тестирование и управление его результатами.

План реализации

Построение системы малыми приращениями

Основной подход

Один билд соответствует реализации одного варианта использования каждым из исполнителей.

Что для этого делаем?

Для этого составляем план итераций в соответствии с рейтингом вариантов использования. Архитектор следит, чтобы между исходниками в проекте и артефактами в моделях была бы ясная взаимосвязь. Необходимо учитывать приоритеты и риски.
По возможности организуем параллельную работу тестеров.

Никакого ручного труда!

Инструменты и практики фазы Construction

1. **Системы контроля версий (SCM).** Это инструменты, которые позволяют отслеживать изменения в коде. Они обеспечивают возможность восстановления предыдущих версий кода, сравнения изменений и совместной работы. Примеры: Git, Mercurial, Subversion.
2. **CI/CD (Continuous Integration/Continuous Delivery).** Это практики, которые позволяют автоматизировать процесс выпуска билдов продукта с оформлением инсталлятора и автоматическим тестированием при каждом изменении в коде. Примеры: Jenkins, GitHub Actions.

Jenkins

Локальный сервер для автоматизации CI/CD

Jenkins — это популярный сервер непрерывной интеграции (CI) с открытым исходным кодом, который помогает автоматизировать сборку, тестирование и развёртывание программного обеспечения.

1. **Гибкость и расширяемость:** поддерживает множество плагинов, которые позволяют интегрировать его с различными системами контроля версий, инструментами тестирования, системами сборки и другими сервисами.
2. **Поддержка различных языков:** поддерживает работу с проектами, написанными на разных языках программирования, таких как Java, Python, JavaScript и др.
3. **Декларативный режим:** конвейер сборки описывается в легкочитаемом виде в `jenkinsfile`.
4. **Параллельное выполнение:** позволяет параллельно выполнять несколько задач, что ускоряет процесс сборки и тестирования.

Jenkinsfile

```
pipeline {
  agent any
  triggers { pollSCM('* * * * *') }

  stages {
    stage('Checkout') {
      steps {
        sh 'git submodule update --init --recursive'
        sh 'git submodule update --checkout -f --recursive
      }
    }

    stage('prepare') {
      steps {
        sh 'rm -rf /build/*'
        dir ('/out') {
          sh "mkdir -p release"
```

GitHub Actions

Облачный сервис для автоматизации CI/CD

GitHub Actions — сервис для автоматизации рабочих процессов, который интегрирован с GitHub и позволяет создавать настраиваемые рабочие процессы для сборки, тестирования, развёртывания и дрпр. задач.

1. **Интеграция с GitHub:** GitHub Actions тесно интегрирован с GitHub, что позволяет использовать события репозитория (push, pull request, issue) в качестве триггеров для рабочих процессов.
2. **Настраиваемые рабочие процессы:** пользователи создают рабочие процессы с помощью YAML-файлов.
3. **Поддержка различных языков и платформ:** GitHub Actions поддерживает работу с проектами, написанными на разных языках и для различных платформ.
4. **Использование контейнеров и виртуальных машин:** рабочие процессы могут выполняться в контейнерах Docker или на виртуальных машинах, что обеспечивает изоляцию и повторяемость среды выполнения.

ActionExample.yml

```yaml
name: learn-github-actions
run-name: ${{ github.actor }} is learning GitHub Actions
on: [push]
jobs:
  check-bats-version:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install -g bats
      - run: bats -v
```

Модель тестирования

Составные части каждого компонента тестирования

1. *Тестовый пример:* исходные данные и ожидаемые результаты.
2. *Процедура тестирования:* описание, как запускать тестовые примеры.
3. *Тестовый компонент:* артефакт для реализации процедуры тестирования.

Классическая основа

Как работает тестер?

От вариантов использования

Тестер в классическом виде ориентируется на модель вариантов использования и обязан убедиться, что продукт в точности соблюдает их.

Типы тестирования

1. Функциональное тестирование.
2. Регрессионное тестирование.
3. Стресс-тестирование.
4. Тестирование безопасности.

Повсеместность

Как сделать идеальный тест?

Критерии всеобщности тестов:

- охват вариантов пользовательской активности;
- доля покрытия тестами исходного кода;
- охват вариантов условий эксплуатации ПО;
- степень погружения тестеров в компоненты проекта;
- частотность проведения тестирования.

Функциональное тестирование

И Selenium как частный его пример

Selenium — это набор инструментов для автоматизации работы с веб-браузерами. Широко используется для функционального тестирования веб-приложений и позволяет имитировать действия пользователя.

1. **Поддержка различных языков:** предоставляет API для разных языков, таких как Java, Python, C#.
2. **Поддержка различных браузеров:** поддерживает работу с популярными браузерами и может использоваться на разных ОС.
3. **Запись и воспроизведение тестов:** позволяет записывать действия пользователя в браузере и воспроизводить их для автоматизации тестирования.
4. **Интеграция с другими инструментами:** можно интегрировать с другими инструментами для тестирования.

Регрессионное тестирование

И JUnit как частный его пример

Регрессионное тестирование — это вид тестирования программного обеспечения, направленный на проверку того, что новые изменения коде не привели к ошибкам в уже существующих функциях. Пример инструментария — JUnit — фреймворк для тестирования Java-приложений, который широко используется для проведения регрессионного тестирования. Он позволяет разработчикам писать и запускать тесты, чтобы убедиться, что код работает правильно и изменения не привели к нежелательным последствиям.

1. **Организация тестов:** тесты можно группировать и организовывать в тестовые классы и пакеты.
2. **Assertions:** JUnit предоставляет набор утверждений (assertions), которые позволяют проверять ожидаемые результаты и сравнивать их с фактическими.
3. **Тестовый запуск:** JUnit поддерживает различные способы запуска тестов, включая выполнение отдельных тестов, тестовых классов или пакетов.

QAOps

Поменяем организационную структуру

Почему разные команды?

Исторически сложилось так, что функция тестеров доверяются отдельным членам команды, которые могут плохо ориентироваться

А что будет, если QA скрестить с DevOps?

1. Процесс тестирования не разделяется с процессом развёртывания.
2. Тестеры в точности знают, в каких условиях эксплуатируется ПО.

Behaviour-Driven Development

Agile для междисциплинарных продуктов

Накладные расходы

Во всех методологиях заказчик и исполнитель говорят на разных языках, и требуется предусматривать «переводчика» в лице системного аналитика.

Унифицируем язык

Предлагается научиться общаться на языках, как можно ближе друг к другу, параллельно развивая инструментарий, подходящий для обработки материалов на таком языке.

Behave

BDD для Python

Каждая функциональная часть тестируется путём формулировки трёх утверждений на естественном языке:

- *Given:* условие для выполнения теста;
- *When:* момент выполнения теста;
- *Then:* результат выполнения теста.

```
Feature: Aircraft operations
Scenario: Takeoff
  Given a airport
  When obtained a command to takeoff
  Then the aircraft flies to its destination
```

Спасибо за внимание!

Всё о курсе: https://marigostra.ru/materials/engineering.html
Канал в Телеграм: https://t.me/MarigostraRu